



Beginner Web Security CTF Lab

Four attack paths. Four secure comparisons.
[CONFIRM STUDENT NAME] • z5671001

A safe lab for learning trust boundaries

GOAL

Build four beginner-friendly web security challenges.

Pair every exploit with a secure comparison.

Outcome: a complete local teaching lab.

LEARNING FLOW

Attempt the vulnerability.

Reveal an artificial flag.

Compare the same input with a safer design.

SCOPE

Runs only on 127.0.0.1.

Uses artificial flags and demo data.

No external target is scanned or attacked.

Four challenges expose one trust problem

01 TRUST

Cookie Role Bypass

The server trusts a client-side role cookie.

Lesson: authorisation decisions must come from server-controlled identity.

02–03 CONTEXT

Reflected XSS
Unsafe HTML rendering.

Path Traversal
Unsafe filename joining.

Lesson: validate and encode input for the exact output or file context.

04 QUERY LOGIC

Simple SQL Injection

The login query is built by concatenating strings.

Lesson: parameterised queries keep user input as data instead of SQL code.

Attack story: reflected XSS

Trust boundary

A search query is read from the URL and rendered with |safe.

```
<script>alert(1)</script>
```

The payload becomes executable content and reveals the challenge flag. In a real application, the same flaw could run script in another user's browser.

Beginner Web Security CTF Lab
[Home](#)

Challenge 2: Reflected XSS Search Page
Search something:
You searched for:

FLAG{xss_needs_output_encoding}
[Return home to view your updated progress.](#)

Attack story: SQL injection

Data became query logic

A login string is concatenated into SQL, so input can change the WHERE clause.

```
admin' OR '1'='1' --
```

The payload bypasses authentication and reveals the flag. The same trust error appears in the cookie and file challenges: user input crosses a boundary without the correct contextual control.

Beginner Web Security CTF Lab

[Home](#)

Challenge 4: Simple SQL Injection

Username:

Password:

Login

Authentication bypass successful. FLAG{sql_queries_need_parameters}

Debug query:

SELECT * FROM users WHERE username = 'student' AND password = '' OR '1'='1'

Hint 1: submit normal values and observe where they appear in the displayed query.

► Show Hint 2

[Return home to view your updated progress.](#)

From vulnerable behaviour to secure design

The defence is context-specific: move authority to the server, encode output, constrain file access, and bind SQL parameters.

VULNERABLE DESIGN

- Trust the role cookie
- Render input with |safe
- Join an arbitrary filename
- Concatenate login SQL

SECURE COMPARISON

- Server-side session and role lookup
- HTML escaping plus restrictive CSP
- Allowlisted filenames and safe base path
- Parameterised SQL query

Iteration produced measurable improvements

BUILD

Four working challenges
Flags and intended paths
Two-level progressive hints

USER TEST

Two participant sessions
Cookie instructions clarified
Progress tracker and
defence unlocks added

FINAL QA

11 automated tests passed
Direct secure-route locking
CSP-compatible shared
stylesheet

●
WEEK 2 PROPOSAL

●
15 JULY

●
18 JULY

From exploit to secure design

Working educational CTF
Attacker + defender analysis
Evidence-driven improvement

Scope boundary
Local demonstration • intentionally vulnerable routes • not production-ready